Chapter 15 Reference

Python Strings & Methods

A Complete Practical Guide to String Creation, Manipulation, Slicing, and 40+ Built-In String Functions with Code Examples.

Python 3.x | 53 Slides | Comprehensive Deck

1. What is a String?

Concept

Definition

A **string** in Python is an immutable sequence of characters enclosed within single (' '), double (" "), or triple quotes (''' ''' / """ """).

Key Properties

- **Immutable:** Elements cannot be modified after creation.
- **Ordered:** Each character has a specific zero-based index.
- **Textual Data:** Stores letters, numbers, spaces, and symbols.

string_basics.py

```python
# Creating strings with different quotes
s1 = 'Hello World'
s2 = "Python Programming"
s3 = """Multi-line
String Example"""

print(type(s1))
print(s1)
print(s2)
```

OUTPUT

```
Hello World
Python Programming
```

2. String Memory & Indexing

Indexing

Positive & Negative Indexing

Python supports dual indexing for every character in a string:

- **Positive Indexing:** Starts at 0 from the left side.
- **Negative Indexing:** Starts at -1 from the right side.

String: "PYTHON"

Pos Index: 0 | 1 | 2 | 3 | 4 | 5

Neg Index: -6 | -5 | -4 | -3 | -2 | -1

indexing.py

```python
text = "PYTHON WORLD"

# Accessing individual characters
print(text[0])     # First character
print(text[7])     # 'W'
print(text[-1])    # Last character
print(text[-5])    # 'W'
```

OUTPUT

```
P
W
D
W
```

3. String Slicing

Slicing

```
string[start : end : step]
```

Extracts a substring starting at index `start` up to position `end - 1` with a given `step`.

- `start`: Default is 0.
- end: Default is length of string.
- `step`: Step increment (negative steps reverse string).

slicing.py

```python
msg = "Independence"

print(msg[0:4])    # Index 0 to 3
print(msg[4:])     # From index 4 to end
print(msg[::2])    # Alternate characters
print(msg[::-1])   # Reverse string
```

OUTPUT

```
Inde
pendence
Idpnece
ecnednepednI
```

4. Escape Sequences

Syntax

Common Escape Characters

Backslash (\) escapes special formatting characters:

- \' : Single quote inside single-quoted string
- \n : Newline insertion
- \t : Tab space
- \\ : Literal backslash

escape.py

```python
# Escaping quotes & newlines
str1 = "It\'s my pen"
str2 = "Line 1\nLine 2"
str3 = "Brand:\tPentonic"

print(str1)
print(str2)
print(str3)
```

OUTPUT

```
It's my pen
Line 1
Line 2
Brand:  Pentonic
```

5. String Operators

Operators

Built-In String Operators

- + (Concatenation): Joins strings together.
- * (Replication): Repeats string multiple times.
- `in` / `not in` (Membership): Checks substring presence.

operators.py

```
s1 = "Hello "
s2 = "World"

print(s1 + s2)       # Concatenation
print("Hi " * 3)      # Replication
print('W' in s2)     # True
print('x' not in s2) # True
```

OUTPUT

```
Hello World
Hi Hi Hi
True
True
```

6. Traversing a String

Control Flow

Iteration Methods

Traversing means visiting each character of a string sequentially using loops:

- **for loop:** Iterates directly character by character.
- **while loop with len():** Iterates via index positions.

traversal.py

```
word = "CODE"

# Traversing with for loop
for char in word:
    print(char, end="-")
print()
```

OUTPUT

```
C-O-D-E-
```

7. Function: len()

Built-in

```
len(string)
```

Description

Returns the total number of characters in the string, including spaces, punctuation, and special symbols.

- **Return Type:** Integer (`int`)
- **Time Complexity:** O(1)

len_example.py

```
text1 = "India !"
text2 = ""

print("Length:", len(text1))
print("Empty Length:", len(text2))
```

OUTPUT

```
Length: 7
Empty Length: 0
```

8. Method: capitalize()

Case Modification

```
string.capitalize()
```

Description

Converts the first character of the string to uppercase and changes all remaining characters to lowercase.

- **Parameters:** None
- **Return Value:** A new capitalized string.

capitalize.py

```python
s = "hELLO pYTHON"
res = s.capitalize()

print("Original:", s)
print("Result  :", res)
```

OUTPUT

```
Original: hELLO pYTHON
Result  : Hello python
```

9. Method: title()

Case Modification

```
string.title()
```

Description

Capitalizes the first character of every word in the string, making all other characters in each word lowercase.

- Words are delimited by spaces, numbers, or symbols.

title.py

```
text = "python programming language"
res = text.title()

print(res)
```

OUTPUT

```
Python Programming Language
```

10. Method: upper()

Case Modification

```
string.upper()
```

Description

Converts all lowercase letters in the string into uppercase letters. Non-alphabetic characters remain unchanged.

- **Return Value:** Uppercase copy of string.

upper.py

```
a = "india !"
res = a.upper()

print("Uppercase:", res)
```

OUTPUT

```
Uppercase: INDIA !
```

11. Method: lower()

Case Modification

```
string.lower()
```

Description

Converts all uppercase letters in the string into lowercase letters.

- Useful for case-insensitive string comparisons.

lower.py

```
a = "INDIA !"
res = a.lower()

print("Lowercase:", res)
```

OUTPUT

```
Lowercase: india !
```

12. Method: casefold()

Case Modification

`string.casefold()`

Description

Converts the string into lowercase aggressively. It is stronger than `lower()` and designed for caseless matching across international characters (e.g., German 'ß' becomes 'ss').

casefold.py

```
german_str = "der Fluß"

print("lower()   :", german_str.lower())
print("casefold():", german_str.casefold())
```

OUTPUT

```
lower()   : der fluß
casefold(): der fluss
```

13. Method: swapcase()

Case Modification

```
string.swapcase()
```

Description

Swaps the case of all alphabetic characters in the string: converts lowercase letters to uppercase and uppercase letters to lowercase.

swapcase.py

```
msg = "PyThOn 3.12"
res = msg.swapcase()

print(res)
```

OUTPUT

```
pYtHoN 3.12
```

14. Method: find()

Search

```
string.find(sub [, start [, end]])
```

Description

Searches for the first occurrence of substring sub. Returns the starting index if found, or -1 if not found.

find.py

```
info = "Jerry is studying in class X !"

print(info.find("is"))
print(info.find("Python")) # Not found
```

OUTPUT

```
6
-1
```

15. Method: rfind()

Search

```
string.rfind(sub [, start [, end]])
```

Description

Searches for the **highest (last) index** of substring sub in the string. Returns -1 if substring is not present.

rfind.py

```
txt = "apple, banana, cherry, apple"

print("First 'apple':", txt.find("apple"))
print("Last  'apple':", txt.rfind("apple"))
```

OUTPUT

```
First 'apple': 0
Last  'apple': 23
```

16. Method: index()

Search

```
string.index(sub [, start [, end]])
```

Description

Identical to `find()`, but raises a `ValueError` exception if the substring is not found instead of returning `-1`.

index.py

```python
txt = "Hello World"

print(txt.index("World"))
# txt.index("Python") → Raises ValueError
```

OUTPUT

```
6
```

17. Method: rindex()

Search

```
string.rindex(sub [, start [, end]])
```

Description

Returns the highest (last) index of substring sub. Raises a `ValueError` if the substring is absent.

rindex.py

```
txt = "cat, dog, cat, bird"

print(txt.rindex("cat"))
```

OUTPUT

```
10
```

18. Method: count()

Search / Statistics

```
string.count(sub [, start [, end]])
```

Description

Returns the total number of non-overlapping occurrences of substring sub in the given range.

count.py

```python
quote = "Practice makes a man perfect!"

print("Count of 'a':", quote.count("a"))
print("Count of 'P':", quote.count("P"))
```

OUTPUT

```
Count of 'a': 3
Count of 'P': 1
```

19. Method: startswith()

Validation

```
string.startswith(prefix [, start [, end]])
```

Description

Returns `True` if string starts with the specified prefix (or a tuple of prefixes); otherwise returns `False`.

startswith.py

```
filename = "report_2026.pdf"

print(filename.startswith("report"))
print(filename.startswith(("img", "report")))
```

OUTPUT

```
True
True
```

20. Method: endswith()

Validation

```
string.endswith(suffix [, start [, end]])
```

Description

Returns `True` if string ends with the specified suffix (or tuple of suffixes); otherwise returns `False`.

endswith.py

```
data = "Hari is a tall boy"

print(data.endswith("boy"))
print(data.endswith("girl"))
```

OUTPUT

```
True
False
```

21. Method: replace()

Modification

```
string.replace(old, new [, count])
```

Description

Replaces occurrences of substring `old` with new. Optional parameter `count` limits maximum replacements.

replace.py

```
old = "My Name is Riya"
new = old.replace("Riya", "Rohan")

print("Old:", old)
print("New:", new)
```

OUTPUT

```
Old: My Name is Riya
New: My Name is Rohan
```

22. Method: split()

Splitting

```
string.split(sep=None, maxsplit=-1)
```

Description

Splits string into a list of substrings using sep as delimiter. Default sep is whitespace.

split.py

```python
text = "Python,Java,C++,Rust"
words = text.split(",")

print(words)
```

OUTPUT

```
['Python', 'Java', 'C++', 'Rust']
```

23. Method: rsplit()

Splitting

```
string.rsplit(sep=None, maxsplit=-1)
```

Description

Splits string starting from the right. Differs from `split()` primarily when `maxsplit` is specified.

rsplit.py

```
path = "usr/local/bin/python"
res = path.rsplit("/", 1)

print(res)
```

OUTPUT

```
['usr/local/bin', 'python']
```

24. Method: splitlines()

Splitting

```
string.splitlines(keepends=False)
```

Description

Splits string at line breaks (`\n`, `\r`, `\r\n`) and returns a list of lines.

splitlines.py

```
poem = "Line 1\nLine 2\nLine 3"
lines = poem.splitlines()

print(lines)
```

OUTPUT

```
['Line 1', 'Line 2', 'Line 3']
```

25. Method: join()

Joining

```
separator.join(iterable)
```

Description

Concatenates elements of an iterable (list, tuple) into a single string using the calling string as delimiter.

join.py

```
items = ["Riya", "Akshat", "Gem"]
result = ", ".join(items)

print(result)
```

OUTPUT

```
Riya, Akshat, Gem
```

26. Method: strip()

Trimming

```
string.strip([chars])
```

Description

Removes leading and trailing characters (default is whitespace) from the string.

strip.py

```
s1 = "   hello world   "
s2 = "**python**"

print(s1.strip())
print(s2.strip("*"))
```

OUTPUT

```
hello world
python
```

27. Method: lstrip()

Trimming

```
string.lstrip([chars])
```

Description

Removes **leading (left-side)** characters/whitespace from the string.

lstrip.py

```
txt = "   python   "

print("'" + txt.lstrip() + "'")
```

OUTPUT

```
'python   '
```

28. Method: rstrip()

Trimming

```
string.rstrip([chars])
```

Description

Removes **trailing (right-side)** characters/whitespace from the string.

rstrip.py

```
txt = "   python   "

print("'" + txt.rstrip() + "'")
```

OUTPUT

```
'   python'
```

29. Method: isalpha()

Testing

```
string.isalpha()
```

Description

Returns `True` if all characters in string are alphabetic letters (A-Z, a-z) and string is non-empty.

isalpha.py

```
s1 = "Python"
s2 = "Python3"

print(s1.isalpha())
print(s2.isalpha())
```

OUTPUT

```
True
False
```

30. Method: isdigit()

Testing

```
string.isdigit()
```

Description

Returns `True` if all characters in the string are digits (0-9, subscripts, superscripts).

isdigit.py

```
n1 = "12345"
n2 = "123a"

print(n1.isdigit())
print(n2.isdigit())
```

OUTPUT

```
True
False
```

31. Method: isalnum()

Testing

```
string.isalnum()
```

Description

Returns `True` if all characters are alphanumeric (letters or numbers) and string is non-empty.

isalnum.py

```
a1 = "Python312"
a2 = "Python 3" # Space included

print(a1.isalnum())
print(a2.isalnum())
```

OUTPUT

```
True
False
```

32. Method: isnumeric()

Testing

```
string.isnumeric()
```

Description

Returns `True` if all characters are numeric (includes digits, fractions, Roman numerals, Chinese numerals).

isnumeric.py

```python
num1 = "123"
num2 = "½" # Unicode fraction

print(num1.isnumeric())
print(num2.isnumeric())
```

OUTPUT

```
True
True
```

33. Method: isdecimal()

Testing

```
string.isdecimal()
```

Description

Returns `True` if all characters are strict decimal digits (base 10 numbers 0-9).

isdecimal.py

```
d1 = "12345"
d2 = "2²" # Superscript

print(d1.isdecimal())
print(d2.isdecimal())
```

OUTPUT

```
True
False
```

34. Method: islower()

Testing

```
string.islower()
```

Description

Returns True if all cased characters in string are lowercase and there is at least one cased character.

islower.py

```
s1 = "hello world"
s2 = "Hello world"

print(s1.islower())
print(s2.islower())
```

OUTPUT

```
True
False
```

35. Method: isupper()

Testing

```
string.isupper()
```

Description

Returns True if all cased characters in string are uppercase and there is at least one cased character.

isupper.py

```
s1 = "PYTHON 3"
s2 = "Python"

print(s1.isupper())
print(s2.isupper())
```

OUTPUT

```
True
False
```

36. Method: isspace()

Testing

```
string.isspace()
```

Description

Returns `True` if there are only whitespace characters (spaces, tabs, newlines) in the string.

isspace.py

```
sp1 = "   \t\n"
sp2 = "  a  "

print(sp1.isspace())
print(sp2.isspace())
```

OUTPUT

```
True
False
```

37. Method: istitle()

Testing

```
string.istitle()
```

Description

Returns True if string is title-cased (each word starts with an uppercase letter followed by lowercase letters).

istitle.py

```
t1 = "Python Programming"
t2 = "python Programming"

print(t1.istitle())
print(t2.istitle())
```

OUTPUT

```
True
False
```

38. Method: isidentifier()

Testing

```
string.isidentifier()
```

Description

Returns `True` if the string is a valid Python identifier (variable/function name rules).

isidentifier.py

```
i1 = "my_var"
i2 = "2nd_var"

print(i1.isidentifier())
print(i2.isidentifier())
```

OUTPUT

```
True
False
```

39. Method: isprintable()

Testing

```
string.isprintable()
```

Description

Returns `True` if all characters in the string are printable (or string is empty). Unprintable chars include `\n`, `\t`.

isprintable.py

```python
p1 = "Hello World!"
p2 = "Hello\nWorld"

print(p1.isprintable())
print(p2.isprintable())
```

OUTPUT

```
True
False
```

40. Method: isascii()

Testing

```
string.isascii()
```

Description

Returns `True` if string is empty or all characters are ASCII (ordinal < 128).

isascii.py

```python
a1 = "Python123"
a2 = "Pythön"

print(a1.isascii())
print(a2.isascii())
```

OUTPUT

```
True
False
```

41. Method: center()

Alignment

```
string.center(width [, fillchar])
```

Description

Centers string in a field of given `width`. Padding is done using `fillchar` (default is space).

center.py

```
title = "PYTHON"

print(title.center(20, "*"))
```

OUTPUT

```
*******PYTHON*******
```

42. Method: ljust()

Alignment

```
string.ljust(width [, fillchar])
```

Description

Left-justifies string in a field of given `width`, padding right side with `fillchar`.

ljust.py

```
word = "Python"

print(word.ljust(15, "-"))
```

OUTPUT

```
Python---------
```

43. Method: rjust()

Alignment

```
string.rjust(width [, fillchar])
```

Description

Right-justifies string in a field of given `width`, padding left side with `fillchar`.

rjust.py

```python
word = "Python"

print(word.rjust(15, "-"))
```

OUTPUT

```
---------Python
```

44. Method: zfill()

Alignment

```
string.zfill(width)
```

Description

Pads string on the left with ASCII `'0'` digits until specified `width` is reached. Handles sign prefix (+/-) correctly.

zfill.py

```
num = "42"
neg = "-42"

print(num.zfill(6))
print(neg.zfill(6))
```

OUTPUT

```
000042
-00042
```

45. Method: format()

Formatting

```
template.format(*args, **kwargs)
```

Description

Formats string by substituting positional and keyword arguments into placeholders {}.

format.py

```
msg = "Hello {}, your score is {:.2f}"
res = msg.format("Akshat", 95.456)

print(res)
```

OUTPUT

```
Hello Akshat, your score is 95.46
```

46. Method: partition()

Splitting

```
string.partition(sep)
```

Description

Splits string at first occurrence of `sep` and returns 3-tuple: `(before, sep, after)`.

partition.py

```python
email = "user@example.com"
parts = email.partition("@")

print(parts)
```

OUTPUT

```
('user', '@', 'example.com')
```

47. Method: rpartition()

Splitting

```
string.rpartition(sep)
```

Description

Splits string at last occurrence of `sep` into a 3-tuple: `(before, sep, after)`.

rpartition.py

```
path = "a/b/c/file.txt"
parts = path.rpartition("/")

print(parts)
```

OUTPUT

```
('a/b/c', '/', 'file.txt')
```

48. Method: removeprefix()

Python 3.9+

```
string.removeprefix(prefix)
```

Description

Removes the given `prefix` if string starts with it; otherwise returns copy of original string.

removeprefix.py

```python
url = "https://python.org"
clean = url.removeprefix("https://")

print(clean)
```

OUTPUT

```
python.org
```

49. Method: removesuffix()

Python 3.9+

```
string.removesuffix(suffix)
```

Description

Removes the given `suffix` if string ends with it; otherwise returns copy of original string.

removesuffix.py

```
fname = "document.pdf"
clean = fname.removesuffix(".pdf")

print(clean)
```

OUTPUT

```
document
```

50. Method: expandtabs()

Formatting

```
string.expandtabs(tabsize=8)
```

Description

Replaces all tab characters `\t` with spaces using given `tabsize`.

expandtabs.py

```
txt = "A\tB\tC"

print(txt.expandtabs(4))
```

OUTPUT

```
A   B   C
```

51. maketrans() & translate()

Translation

```
str.maketrans(x, y) / string.translate(table)
```

Description

`maketrans()` builds a translation mapping table. `translate()` applies the mapping table to replace characters.

translate.py

```
table = str.maketrans("aeiou", "12345")
msg = "hello world"

print(msg.translate(table))
```

OUTPUT

```
h2ll4 w4rld
```

52. Summary & Quick Reference

Cheat Sheet

Category	Methods	Description
Case Conversion	`upper()`, `lower()`, `capitalize()`, `title()`, `swapcase()`	Modify character capitalization.
Search & Count	`find()`, `rfind()`, `index()`, `count()`, `startswith()`, `endswith()`	Find indices & substring counts.
Testing & Validation	`isalpha()`, `isdigit()`, `isalnum()`, `islower()`, `isupper()`, `isspace()`	Return boolean checks.
Splitting & Joining	`split()`, `rsplit()`, `splitlines()`, `join()`, `partition()`	Transform between string & list types.